VISVESVARAYA TECHNOLOGICAL UNIVERSITY

"JnanaSangama", Belgaum -590014, Karnataka.

LAB REPORT

on

# OPERATING SYSTEMS

Submitted by

AKASH (1BM24CS026)

in partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING

B.M.S. COLLEGE OF ENGINEERING

(Autonomous Institution under VTU)

BENGALURU-560019

Feb-2026 to June-2026

B. M. S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering

## CERTIFICATE

This is to certify that the Lab work entitled "OPERATING SYSTEMS – 23CS4PCOPS" carried out by **AKASH(1BM24CS026)**, who is Bonafide student of B. M. S. College of Engineering. It is in partial fulfilment for the award of Bachelor of Engineering in Computer Science and Engineering of the Visvesvaraya Technological University, Belgaum during the year Feb-2026 to June-2026.
The Lab report has been approved as it satisfies the academic requirements in respect of a OPERATING SYSTEMS - (23CS4PCOPS) work prescribed for the said degree.

Dr. Seema Patil
Assistant Professor
Department of CSE
BMSCE, Bengaluru

Dr. Kavitha Sooda
Professor and Head
Department of CSE
BMSCE, Bengaluru

# Index Sheet

## Course Outcomes

| | |
|---|---|
| C01 | Apply the different concepts and functionalities of Operating System |
| C02 | Analyse various Operating system strategies and techniques |
| C03 | Demonstrate the different functionalities of Operating System. |
| C04 | Conduct practical experiments to implement the functionalities of Operating system. |

## Program -1

## Question:

Write a C program to simulate the following non-pre-emptive CPU scheduling algorithm to find turnaround time and waiting time.
a) FCFS
b) SJF
c) Priority
d) Round Robin (Experiment with different quantum sizes
for RR algorithm)

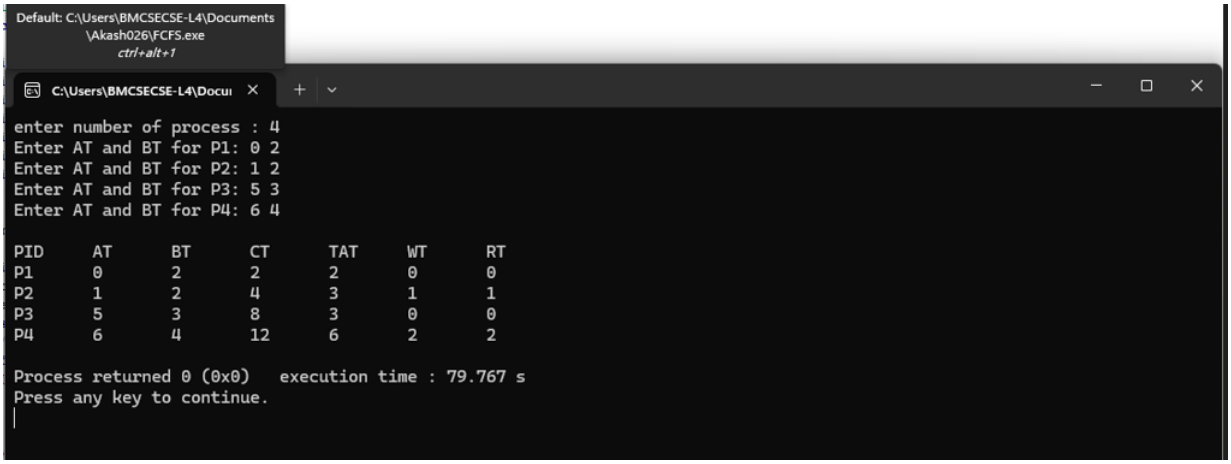
## Code:

### =>FCFS:

```
#include<stdio.h>
struct Process {
   int id, at, bt, ct, tat, wt;
};

void main() {
   int n,current_time=0;
   int sumTat=0, sumWt=0;
   printf("Enter no. of process: ");
   scanf("%d",&n);
   struct Process P[n];
   for(int i=0; i<n; i++) {
      printf("Enter process id: ");
      scanf("%d",&P[i].id);
      printf("Enter arrival time: ");
      scanf("%d",&P[i].at);
      printf("Enter burst time: ");
      scanf("%d",&P[i].bt);
   }

   for(int i=0; i<n-1; i++) {
      for(int j=i+1; j<n; j++) {
         if(P[i].at>P[j].at) {
            struct Process temp=P[i];
            P[i]=P[j];
            P[j]=temp;
         }
      }
```

```
    }

    for(int i=0; i<n; i++) {
        if(current_time<P[i].at)
            current_time=P[i].at;
        P[i].ct=current_time+P[i].bt;
        P[i].tat=P[i].ct-P[i].at;
        P[i].wt=P[i].tat-P[i].bt;
        current_time=P[i].ct;
        sumTat+= P[i].tat;
        sumWt+= P[i].wt;
    }

    printf("PID\tAT \tBT \tCT \tTAT \tWT\n");
    for(int i=0; i<n; i++) {
        printf("%d \t%d \t%d \t%d \t%d \t%d \n",P[i].id,P[i].at,P[i].bt,P[i].ct,P[i].tat,P[i].wt);
    }
    float avgTat= (float)sumTat/n;
    float avgWt= (float)sumWt/n;
    printf("Average TAT time: %f\n",avgTat);
    printf("Average WT time: %f\n",avgWt);
}
```

Result:

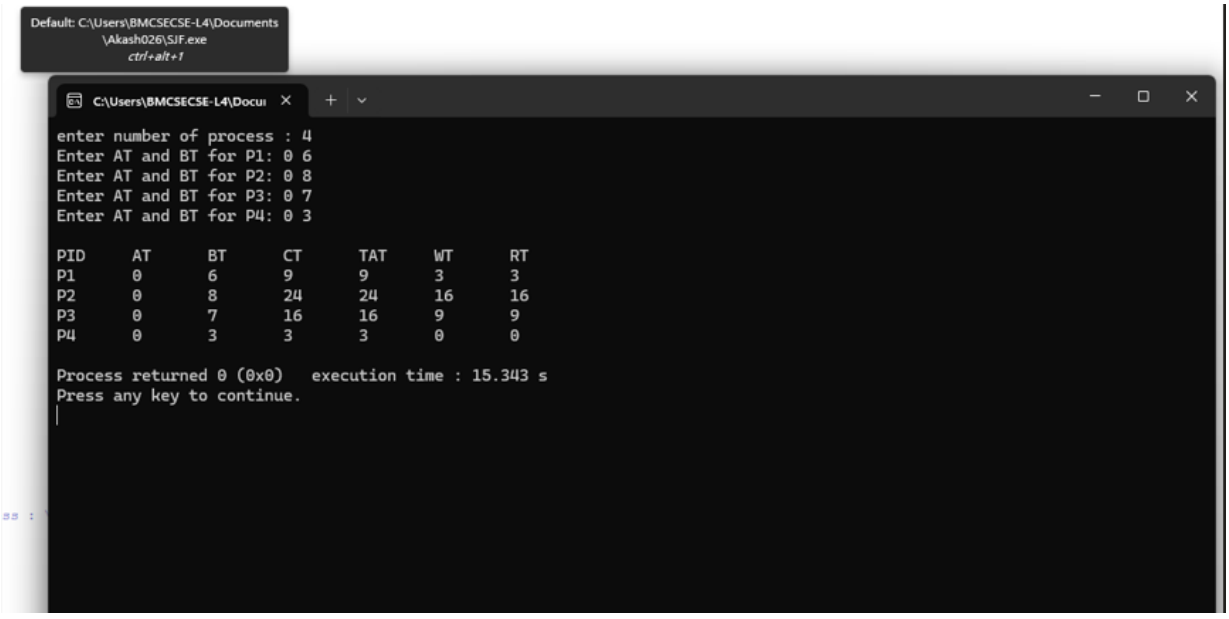


=>SJF(Non-Preemptive):

```
#include <stdio.h>
struct Process {
   int id, at, bt, ct, tat, wt;
};

int main() {
   int n,current_time = 0, completed = 0;
   float avgTat=0, avgWt=0;
   printf("Enter number of processes: ");
   scanf("%d", &n);

   struct Process p[n];
   int visited[n];
   for(int i = 0; i < n; i++) {
      printf("Enter Arrival Time and Burst Time for P%d: ", i+1);
      scanf("%d %d", &p[i].at, &p[i].bt);
      p[i].id = i + 1;
      visited[i] = 0;
   }

   while(completed < n) {
      int idx = -1;
      int min_bt = 9999;
      for(int i = 0; i < n; i++) {
         if(p[i].at <= current_time && visited[i] == 0) {
            if(p[i].bt < min_bt) {
               min_bt = p[i].bt;
               idx = i;
            }
         }
      }

      if(idx != -1) {
         p[idx].ct = current_time + p[idx].bt;
         p[idx].tat = p[idx].ct - p[idx].at;
         p[idx].wt = p[idx].tat - p[idx].bt;
         avgTat += p[idx].tat;
         avgWt += p[idx].wt;
         current_time = p[idx].ct;
         visited[idx] = 1;
         completed++;
```

```
        } else {
            current_time++;
        }
    }

    printf("\nPID\tAT\tBT\tCT\tTAT\tWT\n");
    for(int i = 0; i < n; i++) {
        printf("%d\t%d\t%d\t%d\t%d\t%d\n",p[i].id, p[i].at, p[i].bt,p[i].ct, p[i].tat, p[i].wt);
    }

    avgTat = avgTat/n;
    avgWt = avgWt/n;
    printf("\nAvg TAT : %f",avgTat);
    printf("\nAvg WT : %f",avgWt);

    return 0;
}
```

Result:

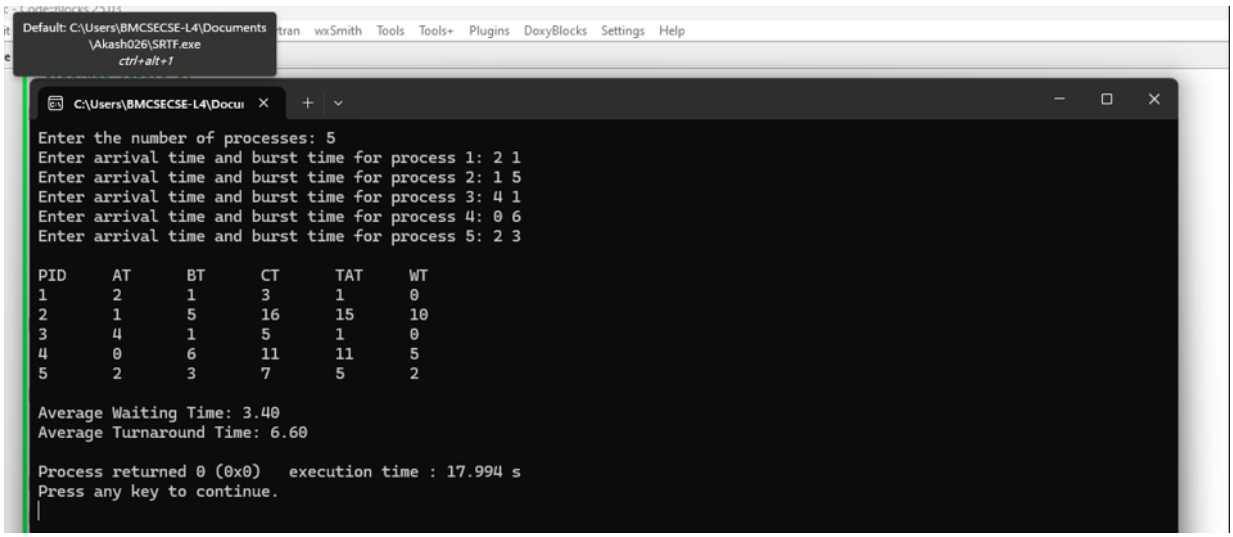


=>SJF(Preemptive)

```
#include <stdio.h>

struct Process {
   int id, at, bt, ct, tat, wt;
};

int main() {
   int n,current_time = 0, completed = 0;
   float avgTat=0, avgWt=0;
   printf("Enter number of processes: ");
   scanf("%d", &n);

   struct Process p[n];
   int remaining[n];
   for(int i = 0; i < n; i++) {
      printf("Enter Arrival Time and Burst Time for P%d: ", i+1);
      scanf("%d %d", &p[i].at, &p[i].bt);
      p[i].id = i + 1;
      remaining[i] = p[i].bt;
   }

   while(completed < n) {
      int idx = -1;
      int min = 9999;
      for(int i = 0; i < n; i++) {
         if(p[i].at <= current_time && remaining[i] > 0) {
            if(remaining[i] < min) {
               min = remaining[i];
               idx = i;
            }
         }
      }

      if(idx != -1) {
         remaining[idx]--;
         current_time++;
         if(remaining[idx]==0) {
            p[idx].ct = current_time;
            p[idx].tat = p[idx].ct - p[idx].at;
            p[idx].wt = p[idx].tat - p[idx].bt;
            avgTat += p[idx].tat;
            avgWt += p[idx].wt;
            completed++;
         }
```

```
        } else {
            current_time++;
        }
    }

     printf("\nPID\tAT\tBT\tCT\tTAT\tWT\n");
    for(int i = 0; i < n; i++) {
        printf("%d\t%d\t%d\t%d\t%d\t%d\n",p[i].id, p[i].at, p[i].bt,p[i].ct, p[i].tat, p[i].wt);
    }

    avgTat = avgTat/n;
    avgWt = avgWt/n;
    printf("\nAvg TAT : %f",avgTat);
    printf("\nAvg WT : %f",avgWt);

    return 0;
}
```

Result:

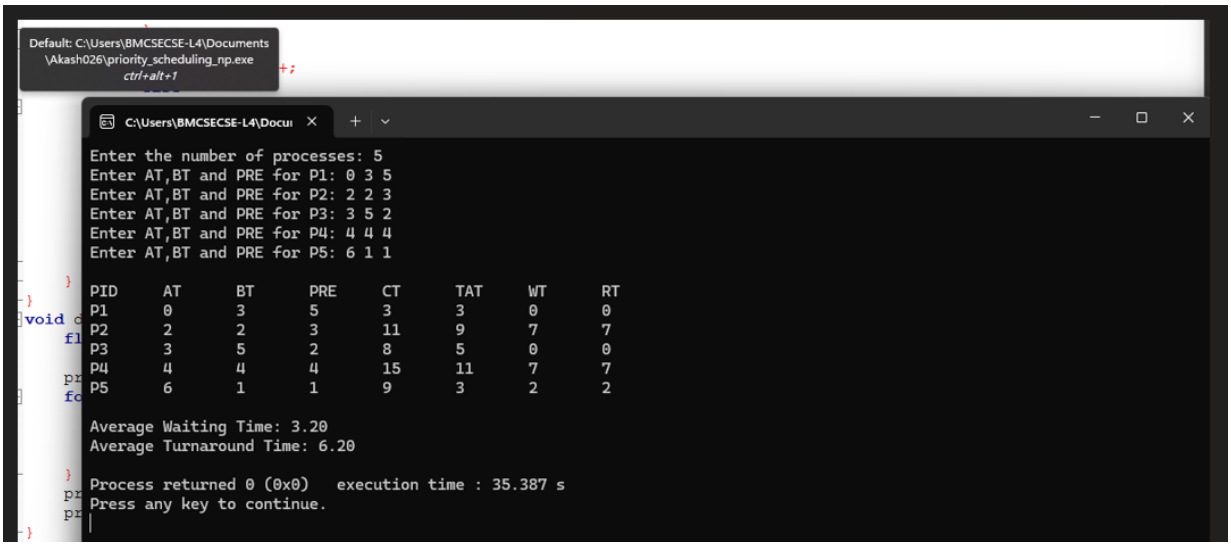


=> Priority (Non-Preemptive)

```
#include<stdio.h>
struct Process {
   int id, at, bt, pr, ct, tat, wt;
};

int main() {
   int n, current_time=0, completed=0;
   float avgTat=0, avgWt=0;
   printf("Enter no. of process: ");
   scanf("%d",&n);
   struct Process P[n];
   int visited[n];

   for(int i=0; i<n; i++) {
      P[i].id=i+1;
      printf("Enter arrival  and burst time for P%d: ",i+1);
      scanf("%d %d",&P[i].at, &P[i].bt);
      printf("Enter priority for P%d: ",i+1);
      scanf("%d",&P[i].pr);
      visited[i]=0;
   }

   printf("Assuming lowest no as highest priority\n");
   while(completed<n) {
      int idx=-1;
      int highest=999;
      for(int i=0; i<n; i++) {
         if(P[i].at<=current_time && visited[i]==0) {
            if(P[i].pr < highest) {
               highest=P[i].pr;
               idx=i;
            }
         }
      }

      if(idx!=-1) {
         current_time += P[idx].bt;
         P[idx].ct =current_time;
         P[idx].tat = P[idx].ct-P[idx].at;
         P[idx].wt = P[idx].tat-P[idx].bt;
         avgTat += P[idx].tat;
         avgWt += P[idx].wt;

         visited[idx]=1;
```

```
            completed++;
        }
        else {
            current_time++;
        }
    }

    printf("\nPID\tAT\tBT\tPR\tCT\tTAT\tWT\n");
    for(int i=0; i<n; i++) {
        printf("%d\t%d\t%d\t%d\t%d\t%d\t%d\n", P[i].id, P[i].at, P[i].bt, P[i].pr, P[i].ct, P[i].tat,
P[i].wt);
    }

    avgTat = avgTat/n;
    avgWt = avgWt/n;
    printf("\nAverage TAT: %f", avgTat);
    printf("\nAverage WT: %f",avgWt);

    return 0;
}
```

Result :

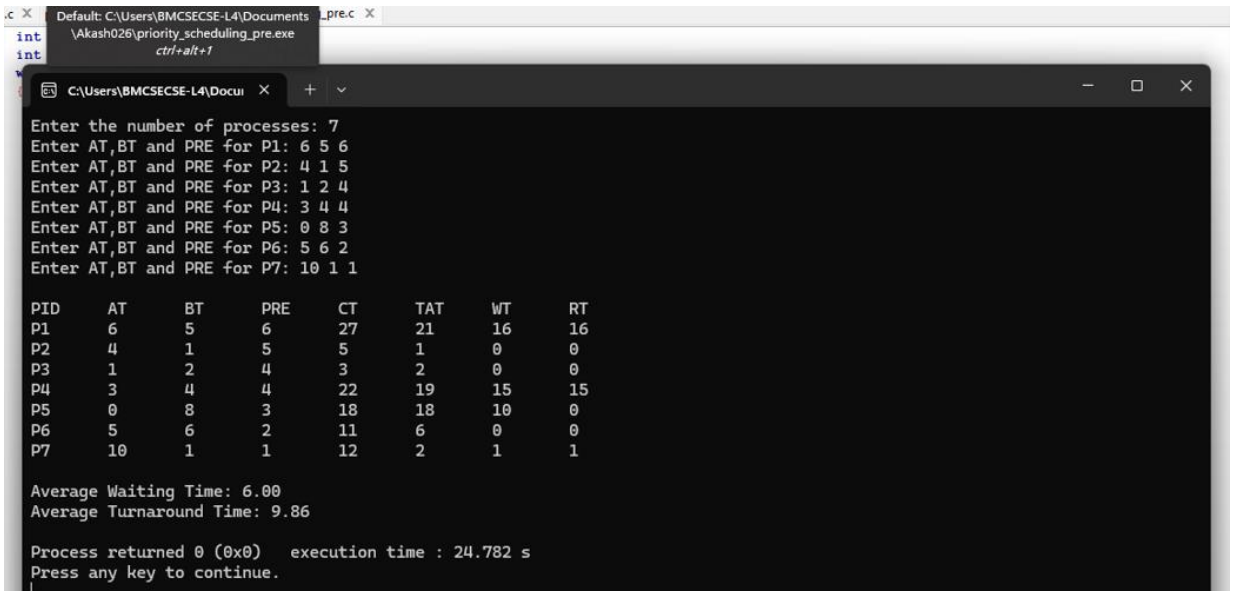


=> Priority (Preemptive)

```
#include<stdio.h>
struct Process {
   int id, at, bt, pr, ct, tat, wt;
};

int main() {
   int n, current_time=0, completed=0;
   float avgTat=0, avgWt=0;

   printf("Enter no. of process: ");
   scanf("%d",&n);
   struct Process P[n];
   int remaining[n];

   for(int i=0; i<n; i++) {
      P[i].id=i+1;
      printf("Enter arrival  and burst time for P%d: ",i+1);
      scanf("%d %d",&P[i].at, &P[i].bt);
      printf("Enter priority for P%d: ",i+1);
      scanf("%d",&P[i].pr);
      remaining[i]=P[i].bt;
   }

   printf("Assuming lowest no as highest priority\n");
   while(completed<n) {
      int idx=-1;
      int highest=999;
      for(int i=0; i<n; i++) {
         if(P[i].at<=current_time && remaining[i]>0) {
            if(P[i].pr < highest) {
               highest=P[i].pr;
               idx=i;
            }
         }
      }

      if(idx != -1) {
         remaining[idx]--;
         current_time++;
         if(remaining[idx]==0) {
            P[idx].ct = current_time;
            P[idx].tat = P[idx].ct - P[idx].at;
            P[idx].wt = P[idx].tat - P[idx].bt;
            avgTat += P[idx].tat;
```

```
            avgWt += P[idx].wt;
            completed++;
        }
    } else {
        current_time++;
    }
}

printf("\nPID\tAT\tBT\tPR\tCT\tTAT\tWT\n");
for(int i=0; i<n; i++) {
    printf("%d\t%d\t%d\t%d\t%d\t%d\t%d\n", P[i].id, P[i].at, P[i].bt, P[i].pr, P[i].ct, P[i].tat,
P[i].wt);
}

avgTat = avgTat/n;
avgWt = avgWt/n;
printf("\nAverage TAT: %f", avgTat);
printf("\nAverage WT: %f",avgWt);

return 0;
}
```

Result :



=>Round Robin

```
#include<stdio.h>

struct Process {
   int id, at, bt, ct, tat, wt, remaining;
};

int main() {
   int n, tq, completed = 0, current_time = 0;
   float avgTat = 0, avgWt = 0;

   printf("Enter number of processes: ");
   scanf("%d",&n);
   struct Process P[n];

   for(int i=0;i<n;i++) {
      P[i].id = i+1;
      printf("Enter arrival and burst time for P%d: ",i+1);
      scanf("%d %d",&P[i].at,&P[i].bt);
      P[i].remaining = P[i].bt;
   }
   printf("Enter Time Quantum: ");
   scanf("%d",&tq);

   for(int i=0;i<n-1;i++) {
      for(int j=i+1;j<n;j++) {
         if(P[i].at > P[j].at) {
            struct Process temp = P[i];
            P[i] = P[j];
            P[j] = temp;
         }
      }
   }

   int queue[100], front = 0, rear = 0;
   int visited[n];
   for(int i=0;i<n;i++)
      visited[i] = 0;
   queue[rear++] = 0;
   visited[0] = 1;

   while(completed < n) {
      int idx = queue[front++];
      if(current_time < P[idx].at)
         current_time = P[idx].at;
```

```
    int exec = (P[idx].remaining > tq) ? tq : P[idx].remaining;
    P[idx].remaining -= exec;
    current_time += exec;

    for(int i=0;i<n;i++) {
        if(visited[i] == 0 && P[i].at <= current_time) {
            queue[rear++] = i;
            visited[i] = 1;
        }
    }

    if(P[idx].remaining > 0) {
        queue[rear++] = idx;
    }
    else {
        P[idx].ct = current_time;
        P[idx].tat = P[idx].ct - P[idx].at;
        P[idx].wt = P[idx].tat - P[idx].bt;
        avgTat += P[idx].tat;
        avgWt += P[idx].wt;

        completed++;
    }

    if(front == rear) {
        for(int i=0;i<n;i++) {
            if(visited[i] == 0) {
                queue[rear++] = i;
                visited[i] = 1;
                current_time = P[i].at;
                break;
            }
        }
    }
}

printf("\nPID\tAT\tBT\tCT\tTAT\tWT\n");
for(int i=0;i<n;i++) {
    printf("%d\t%d\t%d\t%d\t%d\t%d\n",
    P[i].id,P[i].at,P[i].bt,P[i].ct,P[i].tat,P[i].wt);
}
avgTat = avgTat/n;
avgWt = avgWt/n;
printf("\nAverage TAT = %f",avgTat);
```

```
    printf("\nAverage WT = %f",avgWt);

    return 0;
}
```

Result :

```
Enter number of processes: 5
Enter arrival and burst time for P1: 0 8
Enter arrival and burst time for P2: 5 2
Enter arrival and burst time for P3: 1 7
Enter arrival and burst time for P4: 6 3
Enter arrival and burst time for P5: 8 5
Enter Time Quantum: 3

PID     AT      BT      CT      TAT     WT
1       0       8       22      22      14
3       1       7       23      22      15
2       5       2       11      6       4
4       6       3       14      8       5
5       8       5       25      17      12

Average TAT = 15.000000
Average WT = 10.000000
```

## Program – 2

## Question :

Write a C program to simulate multi-level queue scheduling algorithm considering the following scenario. All the processes in the system are divided into two categories – system processes and user processes. System processes are to be given higher priority than user processes. Use FCFS scheduling for the processes in each queue.

## CODE :

```
#include<stdio.h>

struct Process {
   int id, at, bt, type, ct, tat, wt;
   int done;
};

int main() {
   int n;
   printf("Enter number of processes: ");
   scanf("%d",&n);

   struct Process P[n];
   for(int i=0;i<n;i++) {
      P[i].id = i+1;
      printf("Enter arrival and burst time for P%d: ",i+1);
      scanf("%d %d",&P[i].at,&P[i].bt);
      printf("Enter type (1-System, 2-User): ");
      scanf("%d",&P[i].type);
      P[i].done = 0;
   }
   int completed = 0, current_time = 0;
   int sumTat = 0, sumWt = 0;

   while(completed < n) {
      int idx = -1;
      int earliest = 9999;
      for(int i=0;i<n;i++) {
         if(P[i].type == 1 && P[i].done == 0 && P[i].at <= current_time) {
            if(P[i].at < earliest) {
               earliest = P[i].at;
               idx = i;
            }
         }
```

```
        }

        if(idx == -1) {
            earliest = 9999;
            for(int i=0;i<n;i++) {
                if(P[i].type == 2 && P[i].done == 0 && P[i].at <= current_time) {
                    if(P[i].at < earliest) {
                        earliest = P[i].at;
                        idx = i;
                    }
                }
            }
        }

        if(idx != -1) {
            if(current_time < P[idx].at)
                current_time = P[idx].at;
            current_time += P[idx].bt;

            P[idx].ct = current_time;
            P[idx].tat = P[idx].ct - P[idx].at;
            P[idx].wt = P[idx].tat - P[idx].bt;
            sumTat += P[idx].tat;
            sumWt += P[idx].wt;

            P[idx].done = 1;
            completed++;
        }
        else {
            current_time++;
        }
    }

    printf("\nPID\tAT\tBT\tType\tCT\tTAT\tWT\n");
    for(int i=0;i<n;i++) {
        printf("%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
        P[i].id,P[i].at,P[i].bt,P[i].type,
        P[i].ct,P[i].tat,P[i].wt);
    }

    printf("\nAverage TAT = %.2f",(float)sumTat/n);
    printf("\nAverage WT = %.2f",(float)sumWt/n);

    return 0;
}
```

Result :

```
Enter number of processes: 4
Enter arrival and burst time for P1: 0 4
Enter type (1-System, 2-User): 1
Enter arrival and burst time for P2: 0 3
Enter type (1-System, 2-User): 1
Enter arrival and burst time for P3: 0 8
Enter type (1-System, 2-User): 2
Enter arrival and burst time for P4: 10 5
Enter type (1-System, 2-User): 1

PID     AT      BT      Type    CT      TAT     WT
1       0       4       1       4       4       0
2       0       3       1       7       7       4
3       0       8       2       15      15      7
4       10      5       1       20      10      5

Average TAT = 9.00
Average WT = 4.00
```

## Program – 3

## Question:

Write a C program to simulate Real-Time CPU Scheduling algorithms:
a) Rate- Monotonic
b) Earliest-deadline First

## CODE :

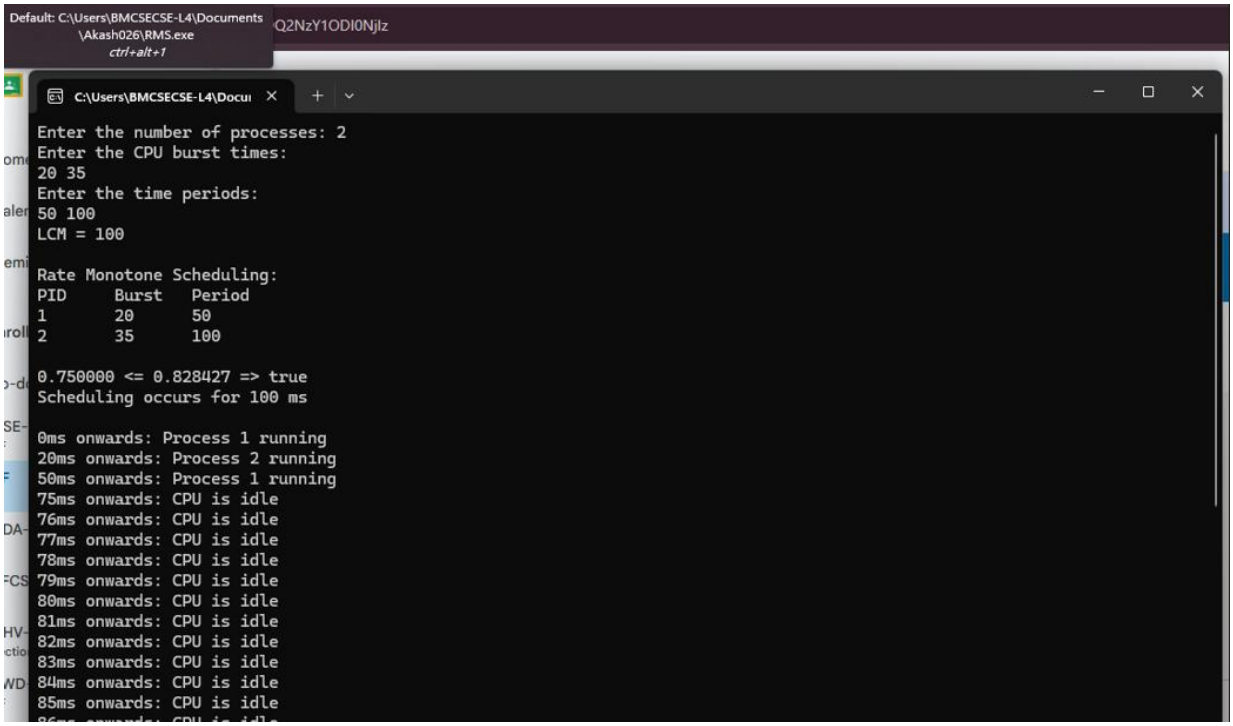
### => Rate- Monotonic

```
#include <stdio.h>
#include <math.h>

struct Process {
    int pid;
    int burst;
    int period;
    int remaining_burst;
};

int gcd(int a, int b) {
    if (b == 0) return a;
    return gcd(b, a % b);
}

int lcm(int a, int b) {
    return (a * b) / gcd(a, b);
}
int main() {
    int n;
    printf("Enter the number of processes: ");
    scanf("%d", &n);

    struct Process p[10];

    printf("Enter the CPU burst times:\n");
    for(int i = 0; i < n; i++) {
        scanf("%d", &p[i].burst);
        p[i].pid = i + 1;
    }

    printf("Enter the time periods:\n");
    int hyperperiod = 1;
    double utilization = 0;
```

```
for(int i = 0; i < n; i++) {
   scanf("%d", &p[i].period);
   p[i].remaining_burst = 0;
   hyperperiod = lcm(hyperperiod, p[i].period);
   utilization += (double)p[i].burst / p[i].period;
}

for(int i = 0; i < n - 1; i++) {
   for(int j = i + 1; j < n; j++) {
      if(p[i].period > p[j].period) {
         struct Process temp = p[i];
         p[i] = p[j];
         p[j] = temp;
      }
   }
}

printf("LCM=%d\n", hyperperiod);
printf("Rate Monotone Scheduling:\n");
printf("PID\tBurst\tPeriod\n");
for(int i = 0; i < n; i++) {
   printf("%d\t%d\t%d\n", p[i].pid, p[i].burst, p[i].period);
}

double bound = n * (pow(2.0, 1.0 / n) - 1.0);
printf("%f<=%f=>%s\n", utilization, bound, (utilization <= bound) ? "true" : "false");

printf("Scheduling occurs for %d ms\n", hyperperiod);

int current_process = -1;

for(int time = 0; time < hyperperiod; time++) {

   for(int i = 0; i < n; i++) {
      if(time % p[i].period == 0) {
         p[i].remaining_burst = p[i].burst;
      }
   }

   int next_process = -1;
   for(int i = 0; i < n; i++) {
      if(p[i].remaining_burst > 0) {
         next_process = i;
         break;
```

```
            }
        }

        if(next_process != current_process) {
            if(next_process == -1) {
                printf("%dms onwards: CPU is idle\n", time);
            } else {
                printf("%dms onwards: Process %d running\n", time, p[next_process].pid);
            }
            current_process = next_process;
        }

        if(current_process != -1) {
            p[current_process].remaining_burst--;
        }
    }

    return 0;
}
```

Result :

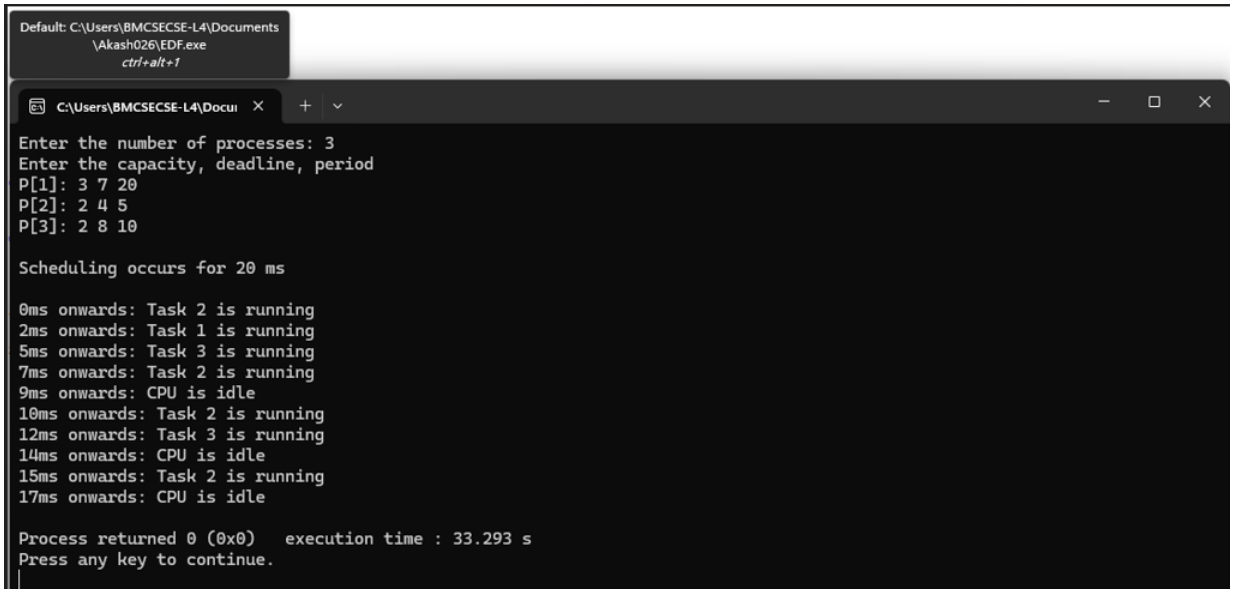


=> Earliest-deadline First

```
#include <stdio.h>
struct Process {
    int pid, burst, deadline, period, remaining_burst, absolute_deadline;
};

int gcd(int a, int b) {
    if (b == 0) return a;
    return gcd(b, a % b);
}

int lcm(int a, int b) {
    return (a * b) / gcd(a, b);
}

int main() {
    int n;
    printf("Enter the number of processes: ");
    scanf("%d", &n);
    struct Process p[10];
    int hyperperiod = 1;
    for (int i = 0; i < n; i++) {
        p[i].pid = i + 1;
        printf("Process %d - Enter Burst, Deadline, Period: ", p[i].pid);
        scanf("%d %d %d", &p[i].burst, &p[i].deadline, &p[i].period);
        p[i].remaining_burst = 0;

        hyperperiod = lcm(hyperperiod, p[i].period);
    }

    printf("\nPID\tBurst\tDeadline\tPeriod\n");
    for (int i = 0; i < n; i++) {
        printf("%d\t%d\t%d\t\t%d\n", p[i].pid, p[i].burst, p[i].deadline, p[i].period);
    }

    printf("Scheduling occurs for %d ms\n", hyperperiod);

    for (int time = 0; time < hyperperiod; time++) {

        for (int i = 0; i < n; i++) {
            if (time % p[i].period == 0) {
                p[i].remaining_burst = p[i].burst;
                p[i].absolute_deadline = time + p[i].deadline;
            }
        }
```

```
        int min_deadline = 999999;
        int selected = -1;
        for (int i = 0; i < n; i++) {
            if (p[i].remaining_burst > 0) {
                if (p[i].absolute_deadline < min_deadline) {
                    min_deadline = p[i].absolute_deadline;
                    selected = i;
                }
            }
        }
        if (selected == -1) {
            printf("%dms: CPU is idle.\n", time);
        } else {
            printf("%dms: Task %d is running.\n", time, p[selected].pid);
            p[selected].remaining_burst--;
        }
    }

    return 0;
}
```

Result :



## Program – 4

## Question :

Write a C program to simulate:
a) Producer-Consumer problem using semaphores.
b) Dining-Philosopher's problem

## Code :

### => Producer-Consumer

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>

#define N 5

int buf[N], in = 0, out = 0;
sem_t empty, full;
pthread_mutex_t m;

void* prod(void* p) {
   int x = 0;
   while (1) {
      sem_wait(&empty);
      pthread_mutex_lock(&m);

      buf[in] = x;
      printf("Produced: %d at buffer[%d]\n", x++, in);
      in = (in + 1) % N;

      pthread_mutex_unlock(&m);
      sem_post(&full);
   }
}

void* cons(void* p) {
   int x;
   while (1) {
      sem_wait(&full);
      pthread_mutex_lock(&m);

      x = buf[out];
      printf("Consumed: %d from buffer[%d]\n", x, out);
      out = (out + 1) % N;
```

```
        pthread_mutex_unlock(&m);
        sem_post(&empty);
    }
}

int main() {
    pthread_t p, c;

    sem_init(&empty, 0, N);
    sem_init(&full, 0, 0);
    pthread_mutex_init(&m, NULL);

    pthread_create(&p, NULL, prod, NULL);
    pthread_create(&c, NULL, cons, NULL);

    pthread_join(p, NULL);
    pthread_join(c, NULL);
}
```

Result :

```
Produced: 0 at buffer[0]
Consumed: 0 from buffer[0]
Produced: 1 at buffer[1]
Produced: 2 at buffer[2]
Consumed: 1 from buffer[1]
Produced: 3 at buffer[3]
Produced: 4 at buffer[4]
Consumed: 2 from buffer[2]
Produced: 5 at buffer[0]
Consumed: 3 from buffer[3]
Produced: 6 at buffer[1]
Produced: 7 at buffer[2]
Produced: 8 at buffer[3]
Consumed: 4 from buffer[4]
Produced: 9 at buffer[4]
Consumed: 5 from buffer[0]
Produced: 10 at buffer[0]
Consumed: 6 from buffer[1]
Produced: 11 at buffer[1]
Consumed: 7 from buffer[2]
Produced: 12 at buffer[2]
Consumed: 8 from buffer[3]
Produced: 13 at buffer[3]
Consumed: 9 from buffer[4]
Produced: 14 at buffer[4]
Consumed: 10 from buffer[0]
Produced: 15 at buffer[0]
Consumed: 11 from buffer[1]
Produced: 16 at buffer[1]
Consumed: 12 from buffer[2]
Produced: 17 at buffer[2]
Consumed: 13 from buffer[3]
Produced: 18 at buffer[3]
Consumed: 14 from buffer[4]
Produced: 19 at buffer[4]
```

=> Dining Philosophers

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

#define N 5

pthread_mutex_t forks[N];

void* phil(void* n) {
    int id = *(int*)n;
    int L = id, R = (id + 1) % N;

    while (1) {
        printf("P%d thinking\n", id);
        sleep(1);

        if (id % 2 == 0) {
            pthread_mutex_lock(&forks[L]);
            pthread_mutex_lock(&forks[R]);
        } else {
            pthread_mutex_lock(&forks[R]);
            pthread_mutex_lock(&forks[L]);
        }

        printf("P%d eating\n", id);
        sleep(2);

        pthread_mutex_unlock(&forks[L]);
        pthread_mutex_unlock(&forks[R]);
    }
}

int main() {
    pthread_t t[N];
    int id[N];

    for (int i = 0; i < N; i++) {
        pthread_mutex_init(&forks[i], NULL);
        id[i] = i;
        pthread_create(&t[i], NULL, phil, &id[i]);
    }

    for (int i = 0; i < N; i++)
```

```
    pthread_join(t[i], NULL);
}
```

Result :

```
Philosopher 0 is thinking.
Philosopher 2 is thinking.
Philosopher 1 is thinking.
Philosopher 4 is thinking.
Philosopher 3 is thinking.
Philosopher 3 picked up right fork 4.
Philosopher 3 picked up left fork 3.
Philosopher 1 picked up right fork 2.
Philosopher 3 is eating.
Philosopher 0 picked up left fork 0.
Philosopher 1 picked up left fork 1.
Philosopher 1 is eating.
Philosopher 1 put down forks 1 and 2.
Philosopher 0 picked up right fork 1.
Philosopher 0 is eating.
Philosopher 4 picked up left fork 4.
Philosopher 2 picked up left fork 2.
Philosopher 2 picked up right fork 3.
Philosopher 2 is eating.
Philosopher 3 put down forks 3 and 4.
Philosopher 3 is thinking.
Philosopher 1 is thinking.
Philosopher 1 picked up right fork 2.
Philosopher 1 picked up left fork 1.
Philosopher 1 is eating.
Philosopher 0 put down forks 0 and 1.
Philosopher 0 is thinking.
Philosopher 4 picked up right fork 0.
Philosopher 4 is eating.
Philosopher 2 put down forks 2 and 3.
Philosopher 2 is thinking.
Philosopher 3 picked up right fork 4.
Philosopher 3 picked up left fork 3.
Philosopher 3 is eating.
Philosopher 1 put down forks 1 and 2.
Philosopher 1 is thinking.
Philosopher 4 put down forks 4 and 0.
Philosopher 4 is thinking.
Philosopher 2 picked up left fork 2.
Philosopher 0 picked up left fork 0.
```

## Program – 5

## Question:

Write a C program to simulate: (Any one)
a) Bankers' algorithm for the purpose of deadlock avoidance.
b) Deadlock Detection

## CODE :

## => Banker's

```
#include<stdio.h>

int main() {
   int n, m;

   printf("Enter number of processes: ");
   scanf("%d",&n);
   printf("Enter number of resource types: ");
   scanf("%d",&m);

   int allocation[n][m], max[n][m], need[n][m];
   int available[m];

   printf("Enter Allocation Matrix:\n");
   for(int i=0;i<n;i++) {
      for(int j=0;j<m;j++) {
         scanf("%d",&allocation[i][j]);
      }
   }

   printf("Enter Max Matrix:\n");
   for(int i=0;i<n;i++) {
      for(int j=0;j<m;j++) {
         scanf("%d",&max[i][j]);
      }
   }

   printf("Enter Available Matrix:\n");
   for(int i=0;i<m;i++) {
      scanf("%d",&available[i]);
   }

   for(int i=0;i<n;i++) {
      for(int j=0;j<m;j++) {
         need[i][j] = max[i][j] - allocation[i][j];
      }
```

```
}

int finish[n], safeSeq[n];
int work[m];

for(int i=0;i<n;i++)
   finish[i] = 0;

for(int i=0;i<m;i++)
   work[i] = available[i];

int count = 0;
while(count < n) {
   int found = 0;
   for(int i=0;i<n;i++) {
      if(finish[i] == 0) {
         int possible = 1;
         for(int j=0;j<m;j++) {
            if(need[i][j] > work[j]) {
               possible = 0;
               break;
            }
         }

         if(possible) {
            for(int j=0;j<m;j++) {
               work[j] += allocation[i][j];
            }
            safeSeq[count++] = i;
            finish[i] = 1;
            found = 1;
         }
      }
   }

   if(found == 0)
      break;
}

if(count == n) {
   printf("\nSystem is in SAFE state\n");
   printf("Safe Sequence: ");
   for(int i=0;i<n;i++) {
      printf("P%d",safeSeq[i]);
```

```
            if(i != n-1)
                printf(" -> ");
        }
    }
    else {
        printf("\nSystem is NOT in safe state");
    }

    return 0;
}
```

Result :

```
Enter number of processes: 5
Enter number of resource types: 3
Enter Allocation Matrix:
0 1 0
2 0 0
3 0 2
2 1 1
0 0 2
Enter Max Matrix:
7 5 3
3 2 2
9 0 2
2 2 2
4 3 3
Enter Available Matrix:
3 3 2

System is in SAFE state
Safe Sequence: P1 -> P3 -> P4 -> P0 -> P2
```

=>Deadlock Detection

```
#include <stdio.h>

int main() {
   int n, m, i, j, k, alloc[10][10], req[10][10], avail[10], f[10]={0}, ans[10], ind=0, flag;

   printf("Enter the number of processes: "); scanf("%d", &n);
   printf("Enter the number of resources: "); scanf("%d", &m);

   printf("Enter the allocation matrix:\n");
   for(i=0; i<n; i++) for(j=0; j<m; j++) scanf("%d", &alloc[i][j]);

   printf("Enter the request matrix:\n");
   for(i=0; i<n; i++) for(j=0; j<m; j++) scanf("%d", &req[i][j]);

   printf("Enter the available resources:\n");
   for(j=0; j<m; j++) scanf("%d", &avail[j]);

   // Deadlock Detection Logic
   for(k=0; k<n; k++) {
      for(i=0; i<n; i++) {
         if(!f[i]) {
            flag=0;
            // Check if process request can be met by available resources
            for(j=0; j<m; j++) {
               if(req[i][j] > avail[j]) flag=1;
            }
            // If yes, process executes and releases its allocated resources
            if(!flag) {
               ans[ind++]=i;
               for(j=0; j<m; j++) avail[j] += alloc[i][j];
               f[i]=1;
            }
         }
      }
   }

   // Output formatting
   if(ind == n) {
      printf("System is in safe state.\nSafe Sequence is: ");
      for(i=0; i<n; i++) {
         printf("P%d%s", ans[i],  " ");
      }
   } else {
      printf("Deadlock Detected in the system!\n");
      printf("The deadlocked processes are: ");
      // Loop through the finish array to find the 0s
```

```
        for(i=0; i<n; i++) {
            if(f[i] == 0) {
                printf("P%d ", i);
            }
        }
        printf("\n");
    }return 0;
}
```

Result :

## Program – 6

## Question :

Write a C program to simulate the following contiguous memory allocation techniques. (Any one)

a) Worst-fit

b) Best-fit

c) First-fit

## CODE :

### => Worst-fit, Best-fit and First-fit

```
#include <stdio.h>

#define MAX 100

void firstFit(int blocks[], int m, int processes[], int n) {
   int allocation[MAX];

   for (int i = 0; i < n; i++)
      allocation[i] = -1;
   for (int i = 0; i < n; i++) {
      for (int j = 0; j < m; j++) {
         if (blocks[j] >= processes[i]) {
            allocation[i] = j;
            blocks[j] = -1;
            break;
         }
      }
   }

   printf("\nFIRST FIT ALLOCATION\n");
   printf("Process No\tProcess Size\tBlock No\n");
   for (int i = 0; i < n; i++) {
      printf("%d\t\t%d\t\t", i + 1, processes[i]);
      if (allocation[i] != -1)
         printf("%d\n", allocation[i] + 1);
      else
         printf("Not Allocated\n");
   }
}

void bestFit(int blocks[], int m, int processes[], int n) {
   int allocation[MAX];
   for (int i = 0; i < n; i++)
```

```
        allocation[i] = -1;

    for (int i = 0; i < n; i++) {
        int best = -1;
        for (int j = 0; j < m; j++) {
            if (blocks[j] >= processes[i]) {
                if (best == -1 || blocks[j] < blocks[best]) {
                    best = j;
                }
            }
        }

        if (best != -1) {
            allocation[i] = best;
            blocks[best] = -1;
        }
    }

    printf("\nBEST FIT ALLOCATION\n");
    printf("Process No\tProcess Size\tBlock No\n");
    for (int i = 0; i < n; i++) {
        printf("%d\t\t%d\t\t", i + 1, processes[i]);
        if (allocation[i] != -1)
            printf("%d\n", allocation[i] + 1);
        else
            printf("Not Allocated\n");
    }
}

void worstFit(int blocks[], int m, int processes[], int n) {
    int allocation[MAX];
    for (int i = 0; i < n; i++)
        allocation[i] = -1;
    for (int i = 0; i < n; i++) {
        int worst = -1;
        for (int j = 0; j < m; j++) {
            if (blocks[j] >= processes[i]) {
                if (worst == -1 || blocks[j] > blocks[worst]) {
                    worst = j;
                }
            }
        }

        if (worst != -1) {
            allocation[i] = worst;
```

```
            blocks[worst] = -1;
        }
    }

    printf("\nWORST FIT ALLOCATION\n");
    printf("Process No\tProcess Size\tBlock No\n");
    for (int i = 0; i < n; i++) {
        printf("%d\t\t%d\t\t", i + 1, processes[i]);
        if (allocation[i] != -1)
            printf("%d\n", allocation[i] + 1);
        else
            printf("Not Allocated\n");
    }
}

int main() {
    int m, n;
    printf("ALOK SAHANI\n1BM24CS031\n");
    printf("Enter number of memory blocks: ");
    scanf("%d", &m);
    int blocks[MAX], blocks1[MAX], blocks2[MAX];

    printf("Enter sizes of memory blocks:\n");
    for (int i = 0; i < m; i++) {
        scanf("%d", &blocks[i]);
        blocks1[i] = blocks[i];
        blocks2[i] = blocks[i];
    }

    printf("Enter number of processes: ");
    scanf("%d", &n);

    int processes[MAX];
    printf("Enter sizes of processes:\n");
    for (int i = 0; i < n; i++) {
        scanf("%d", &processes[i]);
    }

    firstFit(blocks, m, processes, n);
    bestFit(blocks1, m, processes, n);
    worstFit(blocks2, m, processes, n);

    return 0;
}
```

Result :

```
Enter number of memory blocks: 5
Enter sizes of memory blocks:
100 500 200 300 600
Enter number of processes: 4
Enter sizes of processes:
212 417 112 426

FIRST FIT ALLOCATION
Process No      Process Size    Block No
1               212             2
2               417             5
3               112             3
4               426             Not Allocated

BEST FIT ALLOCATION
Process No      Process Size    Block No
1               212             4
2               417             2
3               112             3
4               426             5

WORST FIT ALLOCATION
Process No      Process Size    Block No
1               212             5
2               417             2
3               112             4
4               426             Not Allocated

Process returned 0 (0x0)   execution time : 35.746 s
Press any key to continue.
```

Program -7
Question:

Write a C program to simulate page replacement algorithms.
a) FIFO
b) LRU
c) Optimal

CODE:
=> FIFO, LRU and Optimal

```
#include <stdio.h>

int n, m, p[99];

void disp(int f[]) {
   for (int i = 0; i < n; i++) f[i] == -1 ? printf("- ") : printf("%d ", f[i]);
   printf("\n");
}

void fifo() {
   int f[20], pf = 0, nxt = 0;
   for (int i = 0; i < n; i++) f[i] = -1;
   printf("\n--- FIFO ---\n");
   for (int i = 0; i < m; i++) {
      int hit = 0;
      for (int j = 0; j < n; j++) if (f[j] == p[i]) hit = 1;
      if (!hit) { f[nxt] = p[i]; nxt = (nxt + 1) % n; pf++; }
      printf("Page %d -> ", p[i]); disp(f);D:\useful books
   }
   printf("Total Page Faults: %d\n", pf);
}

void opt() {
   int f[20], pf = 0;
   for (int i = 0; i < n; i++) f[i] = -1;
   printf("\n--- OPTIMAL ---\n");
   for (int i = 0; i < m; i++) {
      int hit = 0;
      for (int j = 0; j < n; j++) if (f[j] == p[i]) hit = 1;
      if (!hit) {
         int rep = 0, far = -1;
         for (int j = 0; j < n; j++) {
            if (f[j] == -1) { rep = j; break; }
            int k;
```

```
            for (k = i + 1; k < m; k++) if (p[k] == f[j]) break;
            if (k > far) { far = k; rep = j; }
          }
          f[rep] = p[i]; pf++;
        }
        printf("Page %d -> ", p[i]); disp(f);
    }
    printf("Total Page Faults: %d\n", pf);
}

void lru() {
    int f[20], t[20] = {0}, pf = 0;
    for (int i = 0; i < n; i++) f[i] = -1;
    printf("\n--- LRU ---\n");
    for (int i = 0; i < m; i++) {
        int hit = -1;
        for (int j = 0; j < n; j++) if (f[j] == p[i]) hit = j;
        if (hit != -1) t[hit] = i;
        else {
            int rep = 0, min = 999;
            for (int j = 0; j < n; j++) {
                if (f[j] == -1) { rep = j; break; }
                if (t[j] < min) { min = t[j]; rep = j; }
            }
            f[rep] = p[i]; t[rep] = i; pf++;
        }
        printf("Page %d -> ", p[i]); disp(f);
    }
    printf("Total Page Faults: %d\n", pf);
}

int main() {
    printf("1BM24CS031\n");
    printf("Enter number of frames: "); scanf("%d", &n);
    printf("Enter number of pages: "); scanf("%d", &m);
    printf("Enter reference string: ");
    for (int i = 0; i < m; i++) scanf("%d", &p[i]);

    fifo();
    opt();
    lru();
    return 0;
}
```

Result:

```
Enter number of frames: 3
Enter number of pages: 21
Enter reference string: 5 0 1 0 2 3 0 2 4 3 3 2 0 2 1 2 7 0 1 1 0

--- FIFO ---
Page 5 -> 5 - -
Page 0 -> 5 0 -
Page 1 -> 5 0 1
Page 0 -> 5 0 1
Page 2 -> 2 0 1
Page 3 -> 2 3 1
Page 0 -> 2 3 0
Page 2 -> 2 3 0
Page 4 -> 4 3 0
Page 3 -> 4 3 0
Page 3 -> 4 3 0
Page 2 -> 4 2 0
Page 0 -> 4 2 0
Page 2 -> 4 2 0
Page 1 -> 4 2 1
Page 2 -> 4 2 1
Page 7 -> 7 2 1
Page 0 -> 7 0 1
Page 1 -> 7 0 1
Page 1 -> 7 0 1
Page 0 -> 7 0 1
Total Page Faults: 11

--- OPTIMAL ---
Page 5 -> 5 - -
Page 0 -> 5 0 -
Page 1 -> 5 0 1
Page 0 -> 5 0 1
Page 2 -> 2 0 1
Page 3 -> 2 0 3
Page 0 -> 2 0 3
Page 2 -> 2 0 3
Page 4 -> 2 4 3
Page 3 -> 2 4 3
Page 3 -> 2 4 3
Page 2 -> 2 4 3
Page 0 -> 2 0 3
Page 2 -> 2 0 3
Page 1 -> 2 0 1
Page 2 -> 2 0 1
Page 7 -> 7 0 1
Page 0 -> 7 0 1
Page 1 -> 7 0 1
Page 1 -> 7 0 1
Page 0 -> 7 0 1
Total Page Faults: 9

--- LRU ---
Page 5 -> 5 - -
Page 0 -> 5 0 -
Page 1 -> 5 0 1
Page 0 -> 5 0 1
Page 2 -> 2 0 1
Page 3 -> 2 0 3
Page 0 -> 2 0 3
Page 2 -> 2 0 3
Page 4 -> 2 0 4
Page 3 -> 2 3 4
Page 3 -> 2 3 4
Page 2 -> 2 3 4
Page 0 -> 2 3 0
Page 2 -> 2 3 0
Page 1 -> 2 1 0
Page 2 -> 2 1 0
Page 7 -> 2 1 7
Page 0 -> 2 0 7
Page 1 -> 1 0 7
Page 1 -> 1 0 7
Page 0 -> 1 0 7
Total Page Faults: 12
```